

boom!

How I Was Built

March 2026

The story of how boom! was built -- from first prompt to production app. A React SPA built through conversations with Claude Code.

The Short Version

boom! is a React single-page application built almost entirely through conversations with Claude Code. The human (O'G) provided direction, design taste, and Dataverse domain knowledge. Claude wrote the code, built the components, and iterated on the design. The result is an internal business tool that manages customer data in Microsoft Dataverse with a custom UI, dark/light theming, and an embedded Copilot Studio AI assistant.

Bottom Line

One person + one AI coding assistant = a production React app with 7 entity pages, a feature-rich dashboard, notes with attachments, dark/light theming, Azure AD auth, and an embedded AI chat widget. No team. No sprints. Just conversations.

The Stack Decision

Why React + Dataverse?

O'G is a Microsoft technical seller. The data already lived in Dataverse -- accounts, contacts, tasks, ideas, impacts, projects, meeting summaries. The question was: build a custom front-end or keep using model-driven apps?

The answer: build a custom React SPA. It serves double duty -- a real daily-driver tool and a reference architecture for enterprise customers evaluating React + Dataverse + Azure Static Web Apps.

Why Fluent UI v9?

Microsoft's own design system. Consistent with the platform story, and the DataGrid component handles tabular data patterns well. The entire app uses Fluent UI -- no mixing of component libraries.

Why Azure Static Web Apps?

Zero server management, global CDN, automatic SSL, SPA routing built in. For a client-side-only app that talks directly to Dataverse, there's no need for a backend server.

How It Was Built

Phase 1: Foundation

Prompt pattern:

| Create a React SPA with MSAL authentication that connects to Dataverse Web API

Result:

App.tsx with provider hierarchy (MSAL -> Theme -> Fluent -> Router), dataverseService.ts with the apiRequest helper, msalConfig.ts with Azure AD settings.

Key decision:

Token provider pattern -- TokenProviderSetup gates the entire app until the Dataverse token is ready. Every API call gets a fresh token.

Phase 2: Entity Pages

Prompt pattern:

| Add a Tasks page with DataGrid showing all action items, with create/edit/view dialogs

Result:

7 entity pages (Accounts, Contacts, Tasks, Ideas, Projects, Impacts, Meeting Summaries) all following the same conventions.

Key decision:

Inline edit pattern -- view dialogs double as edit dialogs via an isEditing toggle. No separate edit page or modal.

Phase 3: Dashboard

Prompt pattern:

| Build a dashboard with quick create, work/personal sections, and bookmarking

Result:

Quick create bar + parking lot strip + ideas strip + personal strip + work tile grid + right sidebar with pinned notes.

Key decision:

Inline view/edit dialogs on the dashboard -- clicking an action item or idea opens its full dialog without navigating away.

Phase 4: Design System

Prompt pattern:

| Custom brand palette, Inter font, dark theme as primary experience

Result:

ogBrand palette based on #4a9eff, Inter everywhere, subtle 1px borders, lowercase monospace headers, accent-colored DataGrid left borders.

Key decision:

Each entity type gets its own accent color (red tasks, purple ideas, amber impacts, etc.). Cards: 8px radius, 1px border, no shadow.

Phase 5: Copilot Integration

Prompt pattern:

| Add a floating chat widget connected to Copilot Studio via Direct Line with SSO

Result:

CopilotChat.tsx with Bot Framework Web Chat, SSO token exchange middleware, rocket icon button.

Key decision:

SSO token exchange so users never see a sign-in card. Rocket icon matches the favicon. Direct Line secret in .env, exchanged for conversation token at runtime.

Phase 6: Notes & Attachments

Prompt pattern:

Add a notes timeline to accounts, tasks, ideas, and projects with file attachments and pinning

Result:

Shared NotesTimeline component with add/delete/pin/download capabilities.

Key decision:

Polymorphic annotations table in Dataverse -- one table handles notes for all entity types via the objectid lookup.

Design Principles

These emerged through building, not from a design doc:

- **Inline over navigate.** Edit in place, create from the dashboard, view details in dialogs. Minimize page transitions.
- **Deactivate, don't delete.** All records set statecode to 1 instead of being deleted. Fetch queries filter by statecode eq 0. Annotations are the exception.
- **Lowercase everything.** Nav labels, panel titles, section headers. The app has a deliberate lowercase aesthetic.
- **Borders over shadows.** Subtle 1px borders define surfaces. No drop shadows, no floating cards.
- **Save feedback.** Every save/update/deactivate shows a spinner and disables the button. Toast notifications on success/error.

Tools Used

Tool	Role
Claude Code	Primary developer -- wrote all application code
VS Code	Editor (via Claude Code CLI)
npm	Package management
React 19	UI framework
TypeScript	Type safety
Fluent UI v9	Component library

MSAL.js	Azure AD authentication
Dataverse Web API	Backend data
Azure Static Web Apps	Hosting
Copilot Studio	AI assistant agent
Bot Framework Web Chat	Chat widget
Inter (Google Fonts)	Typography

Lessons Learned

- **1. Field name gotcha.** Dataverse field names don't match display names. "Task Priority" is tdvsp_priority, not tdvsp_taskpriority. An invalid field in \$select returns a 400 and the whole list appears empty.
- **2. Token provider pattern.** Gating the app behind TokenProviderSetup prevents any API call from firing before auth is ready. Every call gets a fresh token -- no stale token bugs.
- **3. Inline edit simplicity.** One dialog with an isEditing toggle is less code and better UX than separate view/edit modes. But you must reset isEditing and editingId when the dialog closes.
- **4. localStorage is per-browser.** Fine for a single-user tool but wouldn't scale to multi-device. Could be moved to Dataverse user settings if needed.
- **5. Direct Line secrets in SPAs.** The secret is in the JavaScript bundle. Exchanging it immediately for a conversation token limits exposure. Acceptable for an internal tool.
- **6. Dark theme is harder.** Getting surface colors, borders, and text contrast right in dark mode takes more iteration than light mode. The ogDarkTheme tokens required comprehensive overrides.